实验报告

数字逻辑与计算机组成实验

lab09：单周期 RV32I CPU

姓名：　法童

学号：　251250046

邮箱：　251250046@smail.nju.edu.cn

实验时间：　2026-11-06

1 实验目的

1. 理解单周期 RV32I CPU 的数据通路与控制信号：取指、译码、读寄存器、ALU 运算、访存和写回在一个时钟周期内完成。

2. 掌握指令存储器、数据存储器和 MMIO 的组织方式，理解字节寻址、指令格式与立即数扩展。

3. 完成上板验证：按键单步执行，LED 显示 PC 低 16 位，七段数码管显示输入参数和结果。

2 实验原理

2.1 单周期数据通路

单周期 CPU 的一条指令在一个时钟周期内完成。PC 输出指令地址，指令存储器（imem）取出 32 位指令；指令中的 `op`、`funct3`、`funct7` 送入控制单元，产生寄存器写使能、ALU 操作数选择、ALU 操作码、分支类型、访存控制和立即数扩展方式等信号。寄存器堆读出两个源操作数，立即数扩展单元按 I/S/B/U/J 格式生成立即数，ALU 完成加减、逻辑、移位和比较；load/store 通过数据存储器（dmem），写回数据在 ALU 结果和 dmem 读出值之间选择；分支与跳转单元根据 ALU 的比较结果产生下一个 PC。

表 1: CPU 主要模块

模块	作用
`pc`	指令地址，下降沿更新
`imem`	1 KiB 指令 ROM，`addr[9:2]` 索引，`$readmemh` 初始化
`ctrl`	由 `op/func3/func7` 产生全部控制信号
`regfile`	32 个 32 位寄存器，`x0` 恒为 0，下降沿写入
`immext`	I/S/B/U/J 五种立即数扩展
`alu`	加减、与或异或、移位、有符号/无符号比较
`bandj`	根据分支类型、零标志和比较结果产生 PC 选择信号
`dmem`	128 KiB 数据存储器，支持字节、半字、字访存

2.2 指令格式与立即数

RV32I 的六种指令格式中，立即数的位分布不同：I 型取 `instr[31:20]`；S 型拆成 `instr[31:25]` 和 `instr[11:7]`；B 型把 12 位偏移拆到 `instr[31]`、`[7]`、`[30:25]`、

[11:8]；U 型取 instr[31:12] 左移 12 位；J 型把 20 位偏移拆到 instr[31]、[19:12]、[20]、[30:21]。扩展时 I/S/B/J 都复制最高位完成符号扩展，U 型低位补 0。

2.3 上板验证方式

上板验证需要把“程序”和“数据”分开：指令存储器在综合时用 $readmemh 装入机器码；程序通过 MMIO 读取拨码开关、写入结果寄存器。CPU 在 100 MHz 时钟下一条指令只需一个周期，一次按键却持续几十毫秒，因此必须把 CPU 的状态更新冻结，只在按键消抖后的脉冲上放行一个周期，这就是单步执行。七段数码管用 1 kHz 扫描节拍轮流点亮 8 位数字，靠视觉暂留显示完整读数。

3 实验环境与器材

- 开发板：Nexys A7-100T（FPGA 型号 xc7a100tcsg324-1）
- 开发工具：Vivado 2020.2（非工程模式）
- 硬件描述语言：Verilog HDL
- 汇编器：RARS（或工程内的 asm.py）生成 inst.hex

4 设计思路

4.1 系统结构

整个工程由时基、按键消抖、单周期 CPU、指令存储器、数据存储器、MMIO 和显示七部分组成。

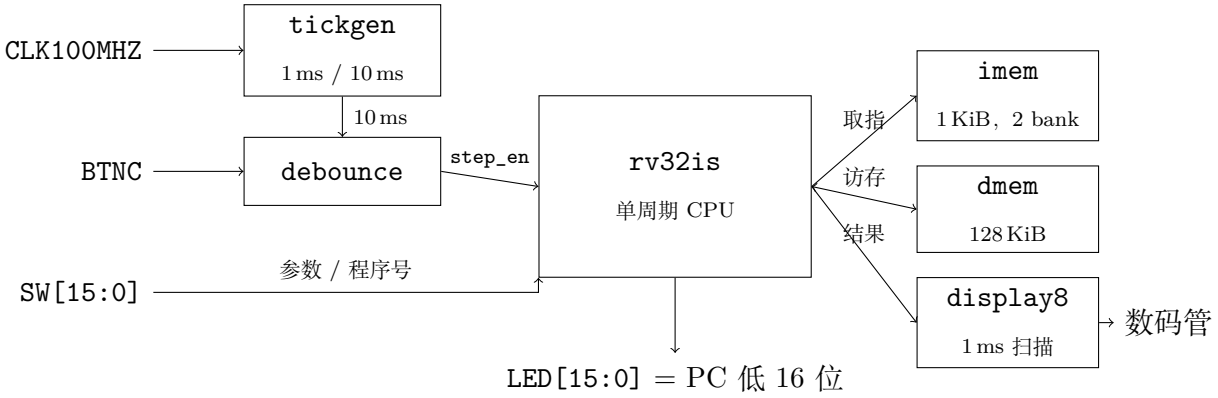


图 1: 单周期 RV32I CPU 上板工程的系统框图

4.2 指令存储器与程序排布

指令存储器共 256 个字（1 KiB），分成两个 bank，每个 bank 128 条指令（512 B）。`SW[13]` 选择程序，送入指令存储器的地址是

```
imemaddr_bank = {22'b0, SW[13], imemaddr[8:0]},
```

存储器内部用 `addr[9:2]` 索引 256 个字。两个程序分别汇编后补齐到 128 字，按顺序拼成 `inst.hex`，综合时由 `$readmemh("inst.hex", ram)` 装入。

```
reg [31:0] ram[255:0]; // 1KB
initial $readmemh("inst.hex", ram);

always @(posedge rdclk)
    if (en)
        dataout <= ram[addr[9:2]];
```

4.3 数据存储器

数据存储器为 32K×32 位（128 KiB）。为了让 Vivado 推断成 BRAM，读口把整字先寄存到 `raw`，字节/半字的符号或零扩展放到存储器外做；写口按 `memop[1:0]` 选择字节、半字或整字写入。

```
(* ram_style = "block" *)
reg [31:0] ram[32767:0];

always @(posedge rdclk) begin
    raw      <= ram[addr[16:2]];
    addr_lo  <= addr[1:0];
    memop_r  <= memop;
end

always @(*) begin
    case (memop_r)
        3'b000: dataout = {{24{raw[8*addr_lo+7]}}, raw[8*addr_lo +:
            8]};
        3'b001: dataout = {{16{raw[16*addr_lo[1]+15]}}, raw[16*
            addr_lo[1] +: 16]};
        3'b100: dataout = {24'b0, raw[8*addr_lo +: 8]};
        3'b101: dataout = {16'b0, raw[16*addr_lo[1] +: 16]};
```

```
        default: dataout = raw;
    endcase
end
```

这两个演示程序只用寄存器运算和 MMIO，不访问数据存储器；dmem 仍然保留在顶层，保证 CPU 的访存接口完整，并单独做了读写测试。

4.4 单步执行

CPU 在 100 MHz 下一条指令只需 10 ns，而一次按键持续几十毫秒。如果直接让它连续运行，一次按键会执行几百万条指令，所以用使能信号 `en` 把状态更新冻结，只在按键脉冲上放行一个周期。

`en` 需要同时门控三个地方：`pc` 不更新、imem 不重新取指、regfile 不重复写入。只停 `pc` 会让 imem 在两次按键之间提前装入下一条指令；不停 regfile 则会让当前指令每个周期重复写回，像 `addi x1,x1,1` 会在一次按键内以 100 MHz 的速度自增。

```
always @(negedge clock)
    if (reset || en)
        pc <= reset ? 32'd0 : nextpc;

regfile myregfile(... .regwr(regwr & en), ...);

always @(posedge rdclk)
    if (en)
        dataout <= ram[addr[9:2]];
```

按键先经过 10 ms 节拍的两次采样消抖，再取上升沿得到一个 100 MHz 周期的脉冲；复位时 `reset` 让 PC 清 0，同时 imem 用 `step_en | reset` 装入第一条指令。

4.5 MMIO 与显示

程序通过两个地址和外界交互：读 `0x100` 得到 `SW[11:0]`，写 `0x104` 把结果送到结果寄存器。顶层根据 `dmemaddr` 译码，MMIO 地址不写入 dmem，dmem 的读出数据和开关值二选一。

```
wire sel_sw     = (dmemaddr == 32'h0000_0100);
wire sel_result = (dmemaddr == 32'h0000_0104);
assign dmemdataout = sel_sw ? {20'b0, SW[11:0]} : dout;

dmem mymem(... .we(dmemwe & ~sel_sw & ~sel_result));
```

八位数码管显示 {4'h0, SW[11:0], result[15:0]}：左边 4 位是输入参数，右边 4 位是结果。参数本身在开关上也能看到，这样显示同时给出“输入什么、算出什么”。结果是 16 位，斐波那契 n 大于 20 时高位会被截掉。

4.6 演示程序

4.6.1 斐波那契

用迭代法计算 $f(n)$，$f(0)=0$、$f(1)=1$。x5 存 n，x6、x7 分别存 $f(i)$、$f(i+1)$，x8 是循环变量 i：

```
loop:
    beq  x8, x5, done
    add  x9, x6, x7
    add  x6, x0, x7
    add  x7, x0, x9
    addi x8, x8, 1
    jal  x0, loop
done:
    sw   x6, 4(x1)
```

4.6.2 素数判断

先用 n < 2 排除 0 和 1，然后从 i=2 开始试除，直到 $i^2 > n$。由于 RV32I 没有乘法指令，i^2 用增量方式维护：每轮 sq += 2i+1，同时 step 自身加 2。余数用重复减法求出，不需要除法指令。

```
outer:
    blt  x5, x8, prime      # n < i*i -> prime
    add  x10, x0, x5        # rem = n
mod:
    sub  x10, x10, x7       # rem -= i
    bge  x10, x7, mod       # while rem >= i
    beq  x10, x0, done      # rem == 0 -> not prime
    addi x7, x7, 1
    add  x8, x8, x9         # sq += 2i+1
    addi x9, x9, 2
    jal  x0, outer
prime:
    addi x6, x0, 1
```

```
done:
    sw   x6, 4(x1)
```

两个程序都以 `jal x0, halt` 自跳转结束，PC 停住，结果保持在数码管上。

5 实验过程

1. 先完成单周期 CPU 的 `rv32is.v`（PC、imem 接口、regfile、立即数扩展、ALU、分支/跳转、控制单元），并通过 OJ 测试集。
2. 编写 `imem.v` 与 `dmem.v`：imem 为 1 KiB、两 bank，使用 `$readmemh` 初始化；dmem 为 128 KiB，读口寄存整字、扩展放在存储器外，写口支持字节/半字/字。
3. 编写 `fib.s`、`prime.s`，汇编后按 bank 补齐到 128 字，拼成 `inst.hex`。
4. 编写 `top.v`：单步使能、bank 选择、MMIO、结果寄存器、LED 显示 PC、八位数码管显示参数与结果。
5. 编写 `rv32is.xdc`：从根目录的 `nexysa7.xdc` 复制，只取消用到的引脚；`SW[8]`、`SW[9]` 保持 `LVCMOS18`。
6. 先跑 `sim/tb_top.v`：用 `defparam` 把时基调快，逐次按键检查 `fib(10)`、多个素数用例和 dmem 的字节/半字读写。
7. 综合、实现、生成比特流并下载到开发板，按下面的测试步骤完成上板验证。

6 测试方法

如何验证：分三层验证。CPU 核心用 OJ 测试集；上板工程先用 `tb_top` 仿真，直接检查结果寄存器；最后下载到开发板，用按键单步执行，读数码管和 LED。

测试信号的选择：

- **斐波那契取 n=10**：结果 55，数值小、便于在 4 位结果区完整显示。
- **素数覆盖边界和不同因数结构**：2（最小素数）、4（偶数合数）、9（平方数合数）、11（素数）、13（素数）、15（奇数合数）、91（7×13）、97（素数）。既有质数也有合数，既有偶数也有奇数，还包含平方数。
- **单步看 PC**：每按一次 `BTNC`，LED 上的 PC 应该加 4；程序最后停在 `jal x0, halt`，PC 不再变化。n=11 的素数程序大约需要 41 次按键，必须按到 PC 停住，结果才在最后的 `sw` 中写入。

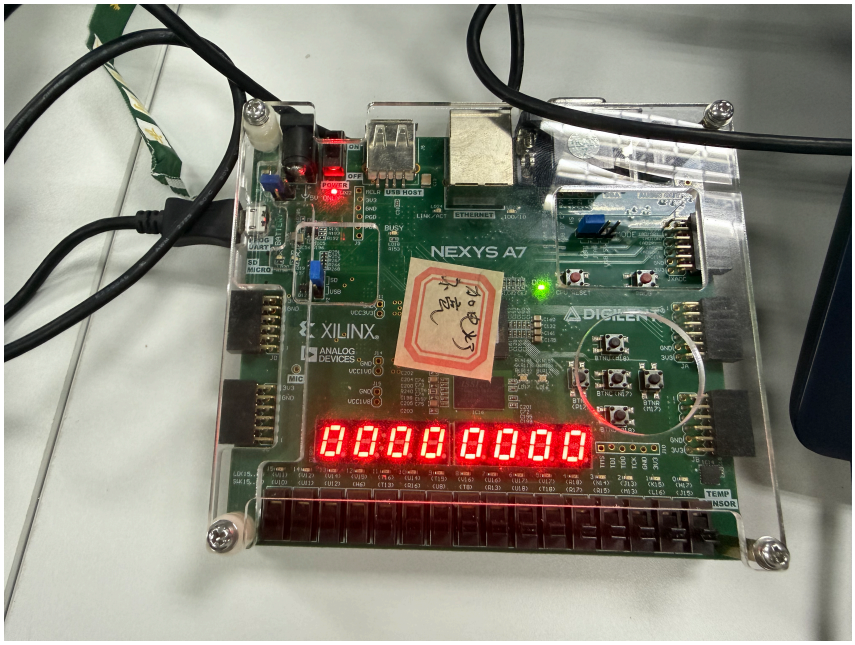


图 2: 下载完成后的初始状态：复位后 PC=0，数码管左四位显示输入参数

- **读数约定**：数码管左边 4 位是参数，右边 4 位是结果；参数用十六进制显示，例如 11 显示为 `000B`。

表 2: 上板测试步骤与预期显示

程序	SW[13]	SW[11:0]	预期显示（参数 结果）
斐波那契 $f(10)$	0	10（000A）	000A0037（55）
素数 11	1	11（000B）	000B0001
素数 13	1	13（000D）	000D0001
素数 15	1	15（000F）	000F0000
素数 91	1	91（005B）	005B0000
素数 97	1	97（0061）	00610001

7 实验结果

7.1 仿真

`tb_top` 中各用例全部通过：dmem 的字、字节、半字读写与符号/零扩展共 7 项；程序层面 `fib(10)=55`，`prime(2)=1`、`prime(4)=0`、`prime(9)=0`、`prime(11)=1`、`prime(13)=1`、`prime(15)=0`、`prime(91)=0`、`prime(97)=1`，与预期一致。

7.2 上板

下载比特流后按上表逐组操作。每按一次 `BTNC` 只执行一条指令，LED 上的 PC 从 0 开始每次加 4；按到 PC 停在 halt 循环后，数码管右边 4 位给出结果，左边 4 位保持当前参数。各组的显示与预期一致，例如素数 11 显示 `000B0001`，素数 15 显示 `000F0000`。

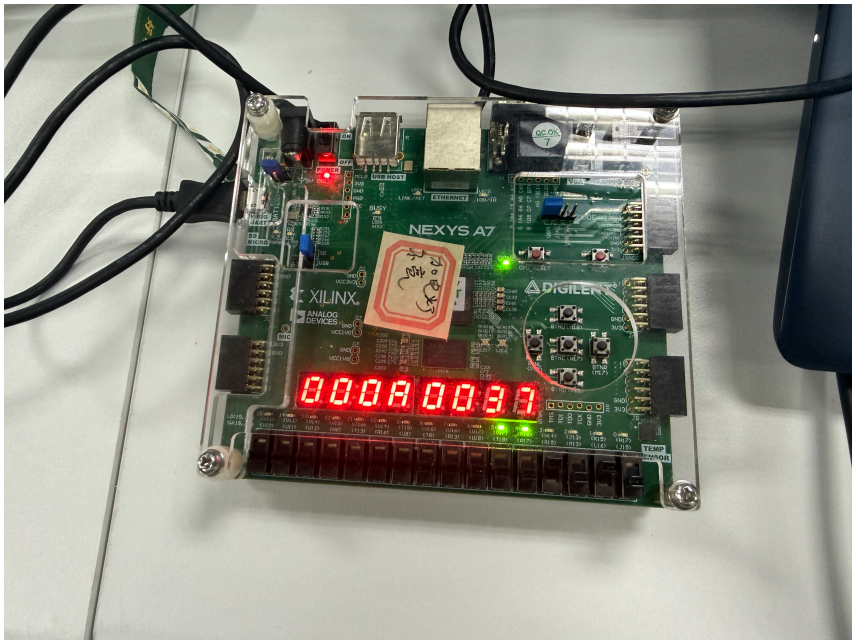


图 3: 上板实测：斐波那契 $n = 10$，显示 000A0037

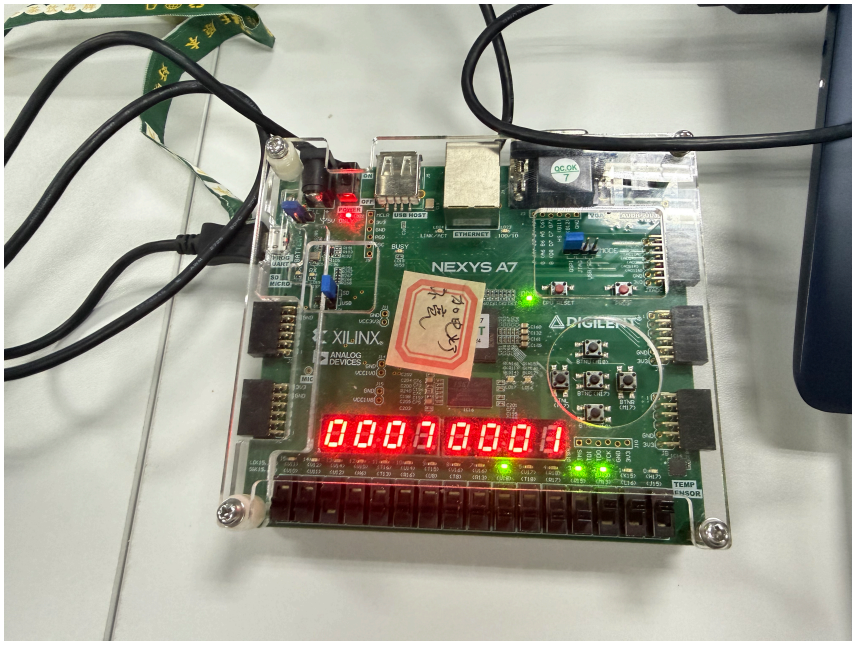


图 4: 上板实测：素数判断，结果在右侧四位

8 问题与解决办法

- **dmem 被综合成分布式 RAM**：最初的读口在同一个 `always` 块里先读整字、再按 `memop` 做符号扩展，Vivado 认为读数据没有直接进寄存器，无法套用 BRAM 模板，于是把 32K×32 拆成 4096 个 `RAM256X1S`，时序直接变成负值。
  解决：把整字先寄存到 `raw`，`addr_lo`、`memop_r` 同步寄存，扩展逻辑放到存储器外；再加 `(* ram_style = "block" *)`，综合后得到 32 个 RAMB36。

- **单步不能只冻结 PC**：只门控 pc 时，`imemaddr` 仍由组合逻辑的 `nextpc` 驱动，两次按键之间 imem 会提前装入下一条指令；不门控 regfile 时，当前指令每个周期重复写回，`addi x1,x1,1` 会一按自增几百万次。
  解决：`en` 同时门控 pc、imem 读和 regfile 写，复位期间用 `reset || en` 让 PC 清 0，imem 用 `step_en | reset` 装入第一条指令。

9 启示与建议

启示：

1. 单周期 CPU 的结构直观，但一条指令的组合路径贯穿取指、译码、寄存器读、ALU 和 nextpc，时序压力集中在一个周期里。上板时用使能冻结状态，既保留了单周期结构，又让每条指令都能被单步观察。

2. 存储器的综合结果取决于读口写法：读数据先寄存、扩展放到存储器外，配合 `ram_style` 才能稳定推断成 BRAM；寄存器堆这种需要组合读的结构则适合分布式 RAM。选哪种资源要服从访问方式，而不是只看容量。

3. 本次实验将此前的所有数据通路组件组装了起来变成了一个完整的通用单周期 CPU，而不再是执行某个特定任务的专用数字电路。在做实验的过程中我也对照了上学期 DLCO 课程中学习的知识和实验中完成的 Logisim 电路设计图，来一步一步完成 CPU 中各个组件、各个执行阶段对应部分数据通路的组装。这个过程还是有不少成就感的，也让我切实的体会到了 HDL 的魅力：虽然我是在 IDE 中编写文字代码，但实际上我在用文字代码去描述、去搭建一套属于自己的硬件系统。

建议：单步执行对长循环不太友好，需要手按很久按钮，可以考虑支持一个“连续执行”按钮或开关，按照特定频率自动顺序执行指令，在保持单步效果的同时省时省力。